

计算机视觉前沿技术报告：多层感知机

课程：计算机视觉前沿技术

姓名：牛晨勋

专业：数据科学与大数据技术

学号：8208231403

1 实验目的

实现一个能识别手写版数字的神经网络。

1. **理解 MLP 原理**：多层感知机是如何工作的，数据怎么在网络里流动、误差怎么传回来修正参数的
2. **NumPy 手搓神经网络**：不使用现成的框架，用 NumPy 实现一个能跑的多层感知机
3. **MNIST 手写数字识别**：训练神经网络学会识别 0-9 这 10 个手写数字
4. **掌握训练技巧**：尝试数据采样、归一化、激活函数、损失函数
5. **可视化训练过程**：画出训练过程中的损失曲线和准确率曲线

2 实验原理

2.1 从线性网络到多层感知机

单层感知机（就是线性分类器）只能解决**线性可分**的问题：一条直线把两类数据分开的情况。

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

但问题是，现实世界很多问题不是线性可分的。比如经典的**异或 (XOR) 问题**：两个输入相同输出 0，不同输出 1。这种问题是很难线性分开的！

而多层感知机（MLP）的核心点是：在输入层与输出层之间**加一层隐藏层**，引入**非线性激活函数**，对数据进行**特征提取和变换**。关键是隐藏层和输出层之间的**非线性激活函数**。如果没有它，不管堆多少层，最后都等价于单层线性网络——因为线性函数的复合还是线性函数。



2.2 我们的网络结构

本实验采用一个 **2 层 MLP** (1 个隐藏层):



输入层有 784 维: 把 28×28 的手写数字图片展平成一个 784 维的向量; **隐藏层则是 256 维**: 提取特征。256 是一个经验值, 既能表达足够复杂的模式, 又不会让计算量太大; **输出层只有 10 维**: 对应 0-9 这 10 个数字, 输出的是每个数字的概率

2.3 激活函数: 给网络加上非线性

1、ReLU (Rectified Linear Unit) 目前最常用的激活函数, 公式简单:

$$\text{ReLU}(x) = \max(0, x)$$

如果输入的是 $x > 0$, 那么输出就是 x 本身; 如果输入 $x \leq 0$, 输出就恒为 0。 **优点**: 计算简单; 缓解梯度消失问题; 收敛速度快。 **缺点**: 负数区域梯度为 0, 可能导致神经元”死亡” (永远输出 0)。

2、Softmax 把网络的原始输出转换成**概率分布**:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

无论输入什么, 所有输出都是正数 (因为 e^x 总是正的), 而且所有输出的和为 1 (可以看作是概率), 而原始得分越高的类别, 输出的概率越大。还有一个**数值稳定性技巧**: 计算时先减去最大值, 可以防止指数爆炸:

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_{j=1}^K e^{z_j - \max(z)}}$$

2.4 损失函数: 衡量预测好坏

这里用**交叉熵损失 (Cross-Entropy Loss)** 来衡量预测结果和真实标签的差距:



$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

$y_{i,c}$ 是真实标签的 one-hot 编码 (若是正确类别为 1, 否则为 0); $\hat{y}_{i,c}$ 是模型预测的概率。只对正确类别的预测概率取对数, 然后求平均。预测越准 (概率接近 1), 损失越小; 预测越错 (概率接近 0), 损失越大

2.5 反向传播: 误差怎么传回去

神经网络训练的**核心**: 首先算输出层的误差, 然后根据链式法则, 一层一层往前传, 算出每个参数对损失的梯度, 用梯度下降更新参数。

输出层梯度: 对于交叉熵损失函数配合 Softmax 激活, 输出层的梯度恰好是预测概率与真实标签的差:

$$\frac{\partial L}{\partial z^{(2)}} = \hat{y} - y$$

隐藏层梯度 (这里需要用链式法则):

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \odot \sigma'(z^{(1)})$$

其中 $\odot$ 表示逐元素相乘, σ' 是激活函数的导数。

参数更新:

$$W := W - \eta \frac{\partial L}{\partial W}$$

沿着梯度的反方向走一小步 (步长由学习率 η 控制)。

2.6 MNIST 数据集

MNIST 是手写数字识别的**标准数据集**, 几乎所有神经网络教程都是用它:

属性	数值
训练集	60,000 张
测试集	10,000 张
图像尺寸	28×28 像素
类别数	10 (数字 0-9)
像素值	0-255 (灰度图)

表 1: MNIST 数据集属性



2.7 数据预处理

MNIST 有 6 万张训练图片，如果全部用 CPU 训练会很慢。我们**每个类别取前 30%**：

- 训练集：约 18000 张（实际 17995 张，每类约 1800 张）
- 验证集：约 3000 张（实际 2996 张，每类约 300 张）

这样既能保证每个数字都有足够的样本（每类 1800 张），又能大大缩短训练时间。原始像素值是 0-255 的整数，我们把它除以 255，映射到 $[0, 1]$ 区间：让梯度更新更稳定，同时也避免数值过大导致计算问题。

3 实验代码

3.1 第 1 步：读取 MNIST 数据

MNIST 数据是二进制格式，需要按照特定格式读取：每张图像是 $28 \times 28 = 784$ 个像素，展平成 784 维向量。

```
1 import numpy as np
2 import struct
3 from array import array
4 # 读取训练标签
5 with open(training_labels_filepath, 'rb') as file:
6     magic, size = struct.unpack(">II", file.read(8))
7     y_train_full = np.array(array("B", file.read()))
8 # 读取训练图片
9 with open(training_images_filepath, 'rb') as file:
10     magic, size, rows, cols = struct.unpack(">IIII", file.
11         read(16))
12     image_data = array("B", file.read())
13     x_train_full = np.array([np.array(image_data[i * 784:(i
14         + 1) * 784])
15         for i in range(size)])
```



3.2 第 2 步：数据可视化

我们随机选了 10 张训练图和 5 张测试图，可以直观地看到手写数字的样子。

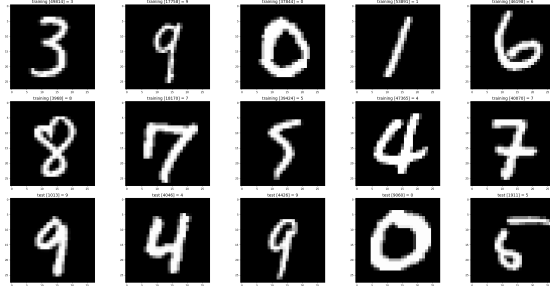


图 1: 手写数字样例图

```
1 def show_images(images, title_texts, save_path):
2     """显示图片列表并保存"""
3     cols = 5
4     rows = int(len(images) / cols) + 1
5     plt.figure(figsize=(30, 20))
6     index = 1
7     for x in zip(images, title_texts):
8         image = x[0]
9         title_text = x[1]
10        plt.subplot(rows, cols, index)
11        plt.imshow(image, cmap=plt.cm.gray)
12        if title_text != '':
13            plt.title(title_text, fontsize=15)
14        index += 1
15    plt.tight_layout()
16    plt.savefig(save_path, dpi=150, bbox_inches='tight')
17    plt.close()
```

3.3 第 3 步：按类别采样 30%

数据集	原始数量	采样后数量	采样比例
训练集	60,000	18,000	30%
验证集	10,000	3,000	30%

表 2: 数据集采样情况

```
1 x_train_list, y_train_list = [], []
2 for cls in range(10): # 遍历 0-9 每个数字类别
3     idx = np.where(y_train_full == cls)[0] # 找到该类的所
        有样本
4     n_sample = int(len(idx) * 0.3) # 计算 30% 的数量
5     x_train_list.append(x_train_full[idx[:n_sample]])
6     y_train_list.append(y_train_full[idx[:n_sample]])
7 x_train = np.vstack(x_train_list) # 合并所有类别
8 y_train = np.hstack(y_train_list)
```

3.4 第 4 步：数据归一化

把像素值从 [0, 255] 映射到 [0, 1]。

```
1 x_train = x_train / 255.0
2 x_val = x_val / 255.0
```

3.5 第 5 步：初始化网络参数

如果权重初始化太大，激活值会爆炸；太小，信号会消失。我们使用的是 **He 初始化**（适用于 ReLU 激活函数），根据输入维度调整随机范围，让信号在网络中可以稳定传播：

$$W \sim \mathcal{N}(0, \frac{2}{n_{in}})$$

如果是 Sigmoid 或 Tanh 激活，应该用 Xavier 初始化 $\frac{1}{n_{in}}$ 。

```
1 np.random.seed(42) # 设置随机种子，保证可复现
2 input_size = 784
3 hidden_size = 256
4 output_size = 10
5 # Xavier/He 初始化
```



```

6 W1 = np.random.randn(input_size, hidden_size) * np.sqrt(2.0
    / input_size)
7 b1 = np.zeros((1, hidden_size))
8 W2 = np.random.randn(hidden_size, output_size) * np.sqrt
    (2.0 / hidden_size)
9 b2 = np.zeros((1, output_size))

```

3.6 第 6 步：定义激活函数和损失函数

交叉熵损失取真实类别对应的预测概率的对数，然后再加一个很小的数 ‘ $1e-12$ ’，防止当预测概率接近 0 时 $\log(0)$ 出现负无穷。

```

1 def relu(z):
2     """ReLU: max(0, z)"""
3     return np.maximum(0, z)
4 def relu_derivative(z):
5     """ReLU 导数"""
6     return (z > 0).astype(float)
7 def softmax(z):
8     """Softmax, 带数值稳定性处理"""
9     z_shift = z - np.max(z, axis=1, keepdims=True)
10    exp_z = np.exp(z_shift)
11    return exp_z / np.sum(exp_z, axis=1, keepdims=True)
12 def cross_entropy_loss(probs, y):
13    """交叉熵损失"""
14    n = len(y)
15    return -np.mean(np.log(probs[np.arange(n), y] + 1e-12))

```

3.7 第 7 步：训练循环（核心代码）

超参数设置：学习率：0.1；批次大小：128；训练轮数：30；优化器：SGD（随机梯度下降）

```

1 epochs = 30
2 lr = 0.1
3 batch_size = 128
4 n_samples = len(x_train)
5 for epoch in range(epochs):

```



```

6      # 打乱数据顺序，让每个 epoch 的训练顺序不同
7      indices = np.random.permutation(n_samples)
8      X_shuffled = x_train[indices]
9      y_shuffled = y_train[indices]
10     # 小批量训练
11     for i in range(0, n_samples, batch_size):
12         X_batch = X_shuffled[i:i + batch_size]
13         y_batch = y_shuffled[i:i + batch_size]
14         # ===== 前向传播 =====
15         z1 = X_batch @ W1 + b1          # 线性变换：输入 ->
            隐藏
16         a1 = relu(z1)                  # ReLU激活
17         z2 = a1 @ W2 + b2              # 线性变换：隐藏 ->
            输出
18         probs = softmax(z2)            # Softmax得到概率
19         # 计算损失
20         loss = cross_entropy_loss(probs, y_batch)
21         # ===== 反向传播 =====
22         n = len(y_batch)
23         y_onehot = np.zeros((n, 10))
24         y_onehot[np.arange(n), y_batch] = 1
25         # 输出层梯度
26         dz2 = (probs - y_onehot) / n
27         dW2 = a1.T @ dz2
28         db2 = np.sum(dz2, axis=0, keepdims=True)
29         # 隐藏层梯度（链式法则）
30         da1 = dz2 @ W2.T
31         dz1 = da1 * relu_derivative(z1)
32         dW1 = X_batch.T @ dz1
33         db1 = np.sum(dz1, axis=0, keepdims=True)
34         # ===== 参数更新（梯度下降） =====
35         W1 -= lr * dW1
36         b1 -= lr * db1
37         W2 -= lr * dW2
38         b2 -= lr * db2

```



3.8 第 8 步：验证和可视化

对验证集做一次前向传播（不需要反向传播），取概率最大的类别作为预测结果，然后计算准确率

```
1 # 验证集准确率
2 z1_val = x_val @ W1 + b1
3 a1_val = relu(z1_val)
4 z2_val = a1_val @ W2 + b2
5 probs_val = softmax(z2_val)
6 val_pred = np.argmax(probs_val, axis=1)
7 val_acc = np.mean(val_pred == y_val)
8 # 绘制训练曲线
9 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
10 ax1.plot(loss_history, 'b-', linewidth=2)
11 ax1.set_title('Training Loss Curve')
12 ax2.plot(np.array(val_acc_history) * 100, 'r-', linewidth
           =2)
13 ax2.set_title('Validation Accuracy Curve')
14 plt.savefig('./MultilayerPerceptron/mlp_training_curves.png', dpi=150)
```



4 实验结果

训练日志（每 5 个 epoch 打印一次）：

Epoch	Loss	Val Acc (%)
5	0.2423	91.29
10	0.1611	92.79
15	0.1154	94.13
20	0.0875	94.43
25	0.0680	94.93
30	0.0542	94.96

表 3: 训练过程记录

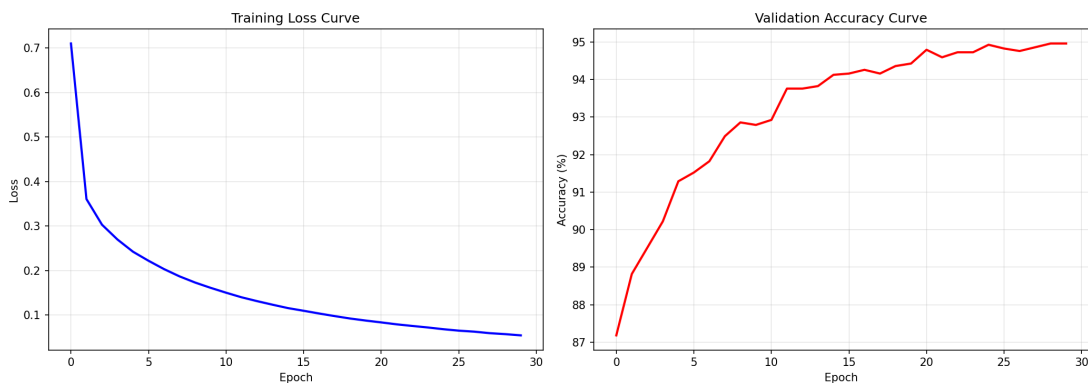


图 2: 训练损失与验证集准确率

5 实验分析

从训练日志和可视化结果可以看出：**损失曲线（左图）**：训练损失从第 5 个 epoch 的 0.24 稳步下降到第 30 个 epoch 的 0.05，模型在不断学习，预测越来越准。损失下降的速度在前 15 个 epoch 较快，后期逐渐放缓，这是正常的——模型接近最优解时，梯度变小，更新幅度也变小。**验证集准确率曲线（右图）**：准确率从第 5 个 epoch 的 91.29% 上升到第 30 个 epoch 的 94.96%，模型泛化能力不断增强。另外，准确率的增长速度也逐渐放缓，从第 25 到 30 个 epoch 只提升了 0.03 个百分点，模型此时已经接近收敛。**过拟合分析**：两条曲线趋势一致，训练损失持续下降的同时准确率也在提升，说明模型没有明显过拟合。如果出现过拟合，我们会看到训练损失继续下降但准确率停滞不前甚至下降，这时就需要添加 Dropout 或 L2 正则化了。

